

АЛГОРИТМИ И СТРУКТУРЕ ПОДАТАКА 2**2025-2026****- трећи домаћи задатак -****Опште напомене:**

1. Домаћи задатак 3 састоји се од једног програмског проблема. Студенти проблем решавају **самостално**, на програмском језику C++.
2. Пре одбране, сви студенти раде тест знања који се ради на рачунару коришћењем система Moodle (<http://elearning.rcub.bg.ac.rs/moodle/>). **Сви студенти треба да креирају налог и пријаве се на курс пре почетка лабораторијских вежби.** Пријава на курс ће бити прихваћена и важећа само уколико се студент региструје путем свог налога електронске поште на серверу **mail.student.etf.bg.ac.rs**.
3. Реализовани програм треба да комуницира са корисником путем једноставног менија који приказује реализоване операције и омогућава сукцесивну примену операција у произвољном редоследу.
4. Унос података треба омогућити било путем читања са стандардног улаза, било путем читања из датотеке.
5. Решења треба да буду отпорна на грешке и треба да кориснику пружи јасно обавештење у случају детекције грешке.
6. Приликом оцењивања, биће узето у обзир рационално коришћење ресурса. **Примена рекурзије се неће признати као успешно решење проблема које може освојити максималан број поена.**
7. Дозвољено је коришћење линеарних структура података из стандардне библиотеке *stl*.
8. За све недовољно јасне захтеве у задатку, студенти треба да усвоје разумну претпоставку у вези реализације програма. Приликом одбране, демонстраторе треба обавестити које претпоставке и која ограничења су усвојена (на пример, максимална димензија низа и слично). Неоправдано увођење ограничавајуће претпоставке повлачи негативне поене.
9. Одбрана трећег домаћег задатка ће се обавити у **уторак, 10.02.2026.** према распореду који ће накнадно бити објављен на сајту предмета.
10. За решавање задатака који имају више комбинација користити следеће формуле.
(**R** – редни број индекса, **G** – последње две цифре године уписа):

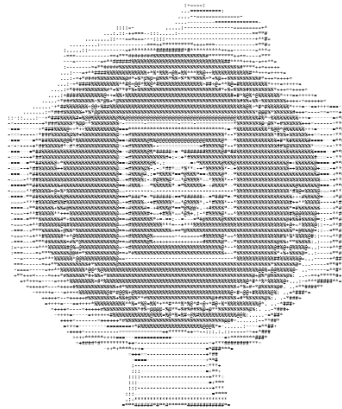
$$i = (R + G) \bmod 2$$

$$j = (R + G) \bmod 3$$

11. Предаја домаћих ће бити омогућена преко Moodle система. Детаљније информације ће бити благовремено објављене. Погрешно назван и предат домаћи задатак повлачи одузимање 25 поена.
12. Предметни наставници задржавају право да изврше проверу сличности предатих домаћих задатака и коригују освојени број поена након одбране домаћих задатака, као и да пријаве теже случајеве повреде Правилника о дисциплинској одговорности студената Универзитета у Београду Дисциплинској комисији Факултета. Током израде решења није дозвољена употреба алата вештачке интелигенције заснованих на великим језичким моделима (*ChatGPT*, *Github Copilot* и сл.).

Задатак 1 – Реконструкција тајне слике употребом алгоритама за сортирање [100 поена]

ASCII уметност (енгл. *ASCII art*) је техника графичког дизајна која подразумева приказивање слике коришћењем текстуалног документа који садржи искључиво карактере који су дефинисани *ASCII* стандардом. На овај начин се свака слика може представити у нешто деградираном облику помоћу $m \cdot n$ карактера, где су m и n дефинисане димензије за генерисање слике.



Слика 1. Лого Електротехничког факултета представљен помоћу *ASCII* карактера

Никола је одлучио да својим колегама пошаље новогодишње честитке које садрже разне сличице направљене помоћу генератора *ASCII* слика, како би уштедео простор уместо слања оригиналних слика. Ипак, одлучио је да направи мали изазов и да оном ко најбрже може да реконструише оригиналну слику да поклон. У наставку је Николина порука:

*„Драге колеге, срећни празници! У прилогу вам шаљем кодирану слику која је добијена мешањем позиција линеаризоване матрице пиксела оригиналне слике. Сваком пикселу, који је представљен једним *ASCII* карактером, је додељен индекс (цео број), тако да су у оригиналној слици, пре мешања, пиксели у низу били уређени растуће на основу овог индекса. Измешао сам пикселе, а ваш задатак је да имплементирате ефикасан алгоритам сортирања ових пиксела који ће вам омогућити да реконструишете слику. Ко имплементира најефикаснији алгоритам добија поклон. Срећно!“*

Никола је послао и кодирану слику у виду текстуалног фајла у следећем формату:

1. Прва линија садржи висину слике, односно број врста у матрици пиксела,
2. Друга линија садржи ширину слике, односно број колона у матрици пиксела и
3. Остале линије садрже за сваки пиксел његов додељени индекс, као и његову *ASCII* репрезентацију, који су одвојени зарезом.

Уз текстуални фајл, Никола је послао помоћне функције и класе за рад са читавањем и обрадом слике, које су његовим колегама на располагању како би се они фокусирали искључиво на имплементацију алгоритама за сортирање.

Маја, Сања, Анастасија, Марко и Матија су одлучили да направе стратегију и да свако од њих осмисли по једну идеју како се низ пиксела може ефикасно сортирати. Сви су дошли до закључка да не треба да користе индивидуалне технике сортирања, већ да је боље употребити хибридне алгоритме. Хибридни алгоритми сортирања користе две или више техника сортирања у сврху постизања бољих перформанси у односу на појединачне технике сортирања.

Марко и Матија су се определили да користе хибридни алгоритам који комбинује сортирање спајањем (енгл. *Merge sort*) и алгоритам директног уметања (енгл. *Insertion sort*), тако што користе сортирање спајањем до одређеног тренутка, а онда користе алгоритам директног уметања за мање партиције.

Маркова хеуристика подразумева сортирање спајањем до тренутка када је партиција која се сортира „скоро сортирана“, након чега се дата партиција сортира директним уметањем. Партиција се сматра „скоро сортираном“ уколико је број инверзија унутар партиције мањи или једнак од $c \cdot n$, где је c константа, а n величина партиције. Број инверзија партиције једнак је броју суседних парова који не задовољавају критеријум уређености (нпр. ако се низ сортира растуће, за свака два суседна елемента партиције за која важи да је први већи од другог се додаје по једна инверзија).

Матијина хеуристика подразумева сортирање спајањем док партиција која се сортира има више од k елемената. Након што број елемената партиције буде k или мање, наставља се сортирање дате партиције директним уметањем.

Са друге стране, Маја, Анастасија и Сања су одлучиле да користе хибридни алгоритам који комбинује партицијско сортирање (енгл. *Quick sort*) и алгоритам директне замене (енгл. *Bubble sort*), тако што користе партицијско сортирање до одређеног тренутка, а онда користе алгоритам директне замене за мање партиције.

Мајина хеуристика подразумева употребу партицијског сортирања до тренутка када дубина рекурзије достигне $c \log_2 n$, где је c константа, а n величина оригиналног низа, након чега се партиција сортира користећи алгоритам директне замене. Дубина рекурзије се овде посматра само концептуално, обзиром да рекурзивна имплементација није дозвољена, те се под дубином рекурзије мисли на број подела партиције који је извршен од почетног низа до посматране партиције.

Сањина хеуристика подразумева употребу партицијског сортирања до тренутка када број поређења елемената низа са пивотом достигне $c \cdot n \log_2 n$, где је c константа, а n величина оригиналног низа, након чега се партиција сортира користећи директну замену.

Анастасијина хеуристика подразумева употребу партицијског сортирања до тренутка када број узастопних лоших подела партиција достигне број k , након чега се наставља са алгоритмом директне замене. Лошом поделом партиције се сматра она подела која испуњава следећи услов, где су L и R величине леве и десне партиције након поделе, респективно, а c константа:

$$\frac{\min(L, R)}{L + R} \leq c$$

Потребно је имплементирати задата два хибридна алгоритма са додељеним хеуристикама и анализирати перформансе у сврху одређивања победника Николиног изазова.

[35 поена] У зависности од редног броја i проблема који се решава, реализовати *Merge sort* и *Insertion sort* хибридни алгоритам који користи:

0. Маркову хеуристику
1. Матијину хеуристику

[35 поена] У зависности од редног броја j проблема који се решава, реализовати *Quick sort* и *Bubble sort* хибридни алгоритам који користи:

0. Мајину хеуристику
1. Сањину хеуристику
2. Анастасијину хеуристику

Сваки алгоритам за сортирање дефинисати новом класом која је изведена из постојеће класе *SortingAlgorithm* и редефинисањем њене методе *sort*. Више информација о начину имплементације алгоритама ће бити дато у Додатку А.

[30 поена] Реконструисати Николине слике и упоредити перформансе имплементираних алгоритама при сортирању са уграђеним алгоритмом сортирања из библиотеке *std*. Потребно је табелирати времена извршавања алгоритама у милисекундама за различите иницијалне поретке пиксела сваке слике. У врстама треба да се нађу Николине слике, а у колонама алгоритми сортирања.

На располагању су класе са методама за читавање кодиране слике у низ пиксела, формирање сортираног низа пиксела у виду слике, као и методе за мерење протеклог времена. Више информација о помоћним методама ће бити дато у Додатку А.

Корисник са програмом интерагује путем једноставног менија. Програм треба да испише садржај менија, а затим да чека да корисник изабере (унесе путем тастатуре) редни број неке од понуђених ставки, након чега, пре извршења, од корисника очекује да по потреби унесе додатне параметре. Поступак се понавља све док корисник у менију не изабере опцију за прекид програма. Све потребне константе за рад алгоритама дефинише корисник путем стандардног улаза.

Напомене

По потреби реализовати и додатне методе, где је то примерено.

За тестирање програма се могу користити датотеке које се налазе у оквиру посебне архиве.

Рад са датотекама у језику C++ захтева увожење заглавља `fstream` (именски простор `std`). За читање података користи се класа `ifstream`. Након отварања датотеке, читање се врши на исти начин као и са стандардног улаза. Кратак преглед најбитнијих метода и пријатељских функција ове класе је дат у наставку.

<pre>void open(const char *_Filename, ios_base::openmode _Mode = ios_base::in, int _Prot = (int)ios_base::_Openprot);</pre>	Отвара датотеку задатог имена за читање. <code>ifstream dat;</code> <code>dat.open("datoteka.txt");</code>
<pre>void close();</pre>	Затвара датотеку.
<pre>bool is_open();</pre>	Утврђује да ли је датотека отворена.
<pre>operator>></pre>	Преклопљен оператор за просте типове података.
<pre>ifstream dat; dat.open("datoteka.dat"); if(! dat.is_open()) greska(); char niz[20]; dat >> niz; dat.close();</pre>	Пример отварања датотеке, провере да ли је отварање успешно, читање једног знаковоног низа из датотеке и затварања датотеке.

Додатак А

Класа **Image** садржи низ пиксела, као и број врста и колона у матрици пиксела оригиналне слике. На располагању су следеће методе ове класе:

- `int getElementCount()` – Дохвата величину низа пиксела;
- `int getElement(int i)` – Дохвата индекс пиксела на позицији *i* у низу;
- `void swapElements(int i, int j)` – Мења места пикселима на позицијама *i* и *j* у низу.

Није могуће директно приступати пољима класе слике, као ни појединачним пикселима у низу, осим кроз претходно наведене методе. Критеријум сортирања пиксела јесте управо индекс који се добија позивом методе `getElement`.

Класа **SortingAlgorithm** је апстрактна класа која дефинише алгоритам сортирања кроз методу `sort`. Ова метода прихвата показивач на слику чији се низ пиксела сортира, као и жељени поредак сортирања (*ASCENDING* за растући поредак, *DESCENDING* за опадајући поредак). Сваки конкретан алгоритам сортирања се дефинише новом класом која треба да буде изведена из ове класе и која треба да редефинише методу `sort`. Дат је пример једног једноставног алгоритма за сортирање класом **ExampleAlgorithm**.

Класа **ImageProcessor** је помоћна класа за рад са учитавањем и исписом слике. Метода која је на располагању је:

- `static void reconstructImage(const char* in_filename, const char* out_filename, SortingAlgorithm* algorithm)`

Ова метода се параметризује називом улазне датотеке која садржи кодирану слику, називом излазне датотеке где ће се реконструисана слика уписати, као и показивачем на објекат конкретног алгоритма за сортирање.

Класа **Stopwatch** је услужна класа за мерење времена. На располагању су следеће методе ове класе:

- `void start()` – Започиње мерење протеклог времена;
- `long long stop()` – Зауставља мерење времена и враћа протекло време у милисекундама.

Пример сегмента програмског кода који врши реконструкцију слике коришћењем алгоритма за сортирање *ExampleSort*, а која се у кодираном облику налази у датотеци *input.txt* и који резултујућу слику испишује у датотеку *output.txt* се налази у наставку:

```
SortingAlgorithm* exampleSort = new ExampleSort();  
ImageProcessor::reconstructImage("input.txt", "output.txt", exampleSort);
```

Није дозвољено мењати класе *Image*, *SortingAlgorithm*, *ImageProcessor* и *Stopwatch*.